

RAG Debugging — Worked Example: Solution

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)
- [Real cause vs. symptom fix](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Basic

Real cause: `text[start:end - 1]` — the `- 1` is an off-by-one slicing mistake. Python slicing (`text[start:end]`) is already exclusive of `end`, so it already stops one character before position `end`; subtracting 1 again drops one extra character from every chunk, at every boundary.

The fix:

```
def chunk_by_chars(text, chunk_size):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end
    return chunks
```

Removing the extra `- 1` makes each chunk exactly `chunk_size` characters (except the last, which can be shorter), with no characters dropped between chunks.

Test that would have caught it:

```
def test_chunk_by_chars_reproduces_the_original_text():
    text = "The quick brown fox jumps over the lazy dog"
    chunks = chunk_by_chars(text, chunk_size=10)
    rebuilt = "".join(chunks)
    assert rebuilt == text
```

Checking that the chunks, joined back together, exactly reproduce the input is a strong, simple check for any chunking function — it catches dropped characters, duplicated characters, and off-by-one errors in one assertion.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Intermediate

Real cause: documents were embedded with one model (`text-embedding-3-small`) and questions are now embedded with a different one (`text-embedding-3-large`). Different embedding models don't share the same number space at all — comparing a `-large` question vector against `-small` document vectors is comparing two unrelated coordinate systems, so every similarity score comes back low and meaningless, no matter how relevant the actual text is. Nothing raises an exception because both models return valid-looking vectors — the failure is purely semantic, not a Python error, which is exactly why it first looks like a generation-quality problem instead of a config mismatch.

The fix:

```
EMBED_MODEL = "text-embedding-3-small"

def embed_documents(chunks):
    return embedding_client.embed(chunks, model=EMBED_MODEL)

def embed_query(question):
    return embedding_client.embed([question], model=EMBED_MODEL)
```

One shared constant, used by both the indexing path and the query path, makes it structurally impossible for them to drift apart again.

Test that would have caught it:

```
def test_index_and_query_use_the_same_embedding_model():
    assert INDEX_EMBED_MODEL == QUERY_EMBED_MODEL
```

A more end-to-end version: embed one known document and a question that clearly matches it, and assert the similarity score is above a reasonable threshold (like 0.5) — a suspiciously low score across every question, not just one, is the signal to check embedding model consistency before anything else.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Real-world

Real cause: fixed-size chunking cut a sentence in half, right at the exact spot containing the fact the question needed. Retrieval technically "worked" — both halves came back in the top results — but generation treated each chunk as a mostly-independent piece of context and never reliably stitched the sentence back together across the boundary. This is the retrieval-vs-generation distinction from Core Concepts, but with a twist: retrieval and chunking are actually the same failure here, one step upstream of where the symptom (a confused-sounding generation answer) shows up.

The fix:

```
def chunk_by_chars_with_overlap(text, chunk_size, overlap):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end - overlap
    return chunks
```

Adding overlap between consecutive chunks means a sentence that falls near a boundary is very likely to appear whole in at least one chunk, instead of only ever appearing split across two. Splitting by paragraph (Doc08's Intermediate exercise) is the more thorough fix, since it respects natural sentence/paragraph breaks instead of cutting at an arbitrary character count at all.

Test that would have caught it:

```
def test_answer_spanning_a_chunk_boundary_is_still_findable():
    document = make_document_with_answer_at_position(char_offset=97, chunk_size=100)
    chunks = chunk_by_chars_with_overlap(document, chunk_size=100, overlap=20)
    found_whole_sentence = False
    for chunk in chunks:
        if "Customers have 30 days" in chunk:
            found_whole_sentence = True
    assert found_whole_sentence
```

This test deliberately places the answer right at a chunk boundary — this is Doc08's own "Edge cases" exercise (`chunk_boundary_split`), and it's exactly the kind of test that won't get written

unless you've already been bitten by this once.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Multi-agent

Real cause: the Research agent's shared-state field only carries a plain-text summary, dropping the score and source metadata the retriever actually returned. The Analysis agent's "flag unsourced claims" check is working exactly as designed — it just never receives the sourcing information it needs to do its job, because that information was thrown away one node earlier. From the outside this looks like a RAG hallucination-adjacent problem ("the system is unsure about something it should know"), but the retrieval and generation were both correct — the bug is entirely in what one agent chose to write into shared state.

The fix:

```
class SharedState(TypedDict):
    task: str
    research_findings: str
    research_sources: list[dict]  # [{"text": ..., "score": ..., "source": ...},

def research_node(state):
    results = retrieve(state["task"], k=3)
    summary = summarize_findings(results)
    sources = []
    for result in results:
        sources.append({
            "text": result["text"],
            "score": result["score"],
            "source": result["source"],
        })
    return Command(goto="analysis_agent", update={
        "research_findings": summary,
        "research_sources": sources,
    })
```

Adding a dedicated `research_sources` field to shared state, alongside the plain-text summary, gives the Analysis agent the actual evidence it needs to verify a claim, instead of forcing it to guess from prose alone.

Test that would have caught it:

```
def test_research_findings_carry_source_metadata_to_analysis(fake_retriever):
    fake_retriever.returns([{"text": "Customers have 30 days...", "score": 0.91,
    result = research_node({"task": "refund window"})
    sources = result.update("research_sources")
    assert len(sources) == 1
    assert sources[0]["source"] == "policy.md"
```

Testing the Research node's *shared-state update*, not just whether the retriever itself found the right chunk, is what catches this — the retriever was never broken, so a test only of `retrieve()` would keep passing forever while this bug shipped.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Real cause vs. symptom fix

A symptom fix would be rewriting the generation prompt to "try harder to combine related chunks" (Round 3) instead of fixing the chunking that split the sentence in the first place, or telling the Analysis agent to "trust research_findings even without sources" (Round 4) instead of actually wiring the sourcing data through. Both make today's specific test case pass while leaving the structural problem in place — a differently-worded document would still split badly, and an unrelated claim with no real source would now get trusted just as blindly as a well-sourced one. The whole point of Doc08's Core Concepts — telling a retrieval failure from a generation failure — is what makes these fixes findable at all: Round 2's failure looked like generation, but the real layer was embeddings; Round 4's failure looked like generation trustworthiness, but the real layer was a missing field in shared state one hop upstream.